CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 10: CALL

Instructor: Justin Yokota

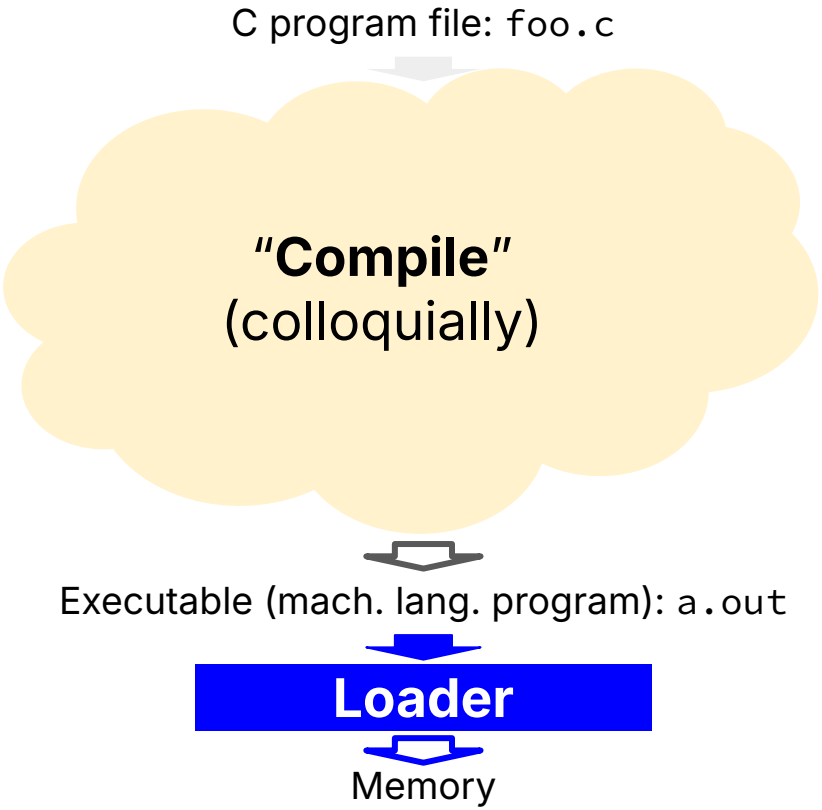
Agenda

- CALL: Compile, Assemble, Link, Load
- Compiler
- Assembler
  - Machine Code
  - Object File Format
- Linker
  - Resolve References
- Loader
- Program Translation vs. Interpretation

Agenda

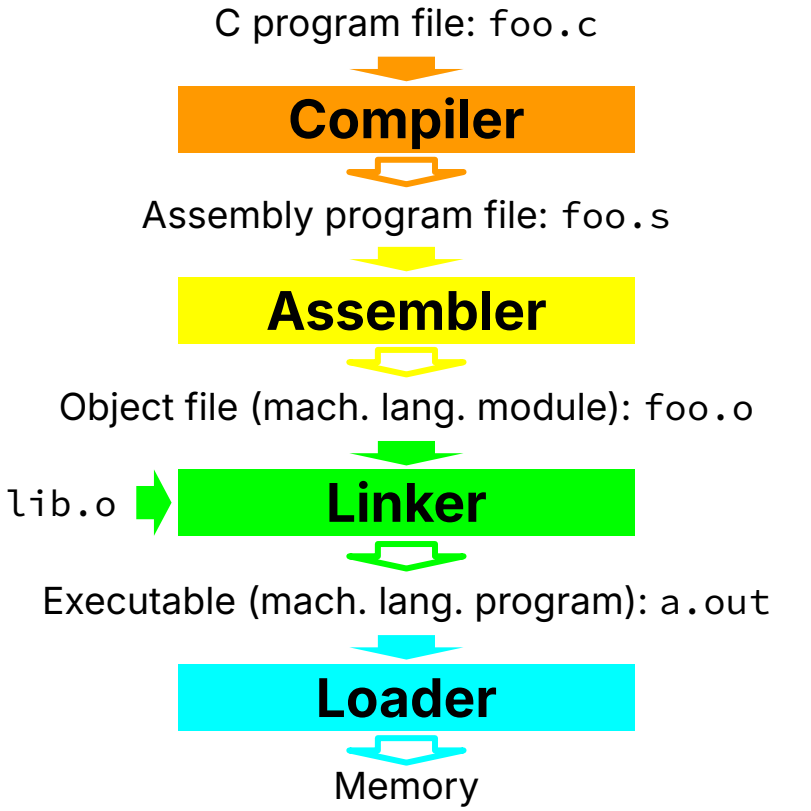
- **CALL: Compile, Assemble, Link, Load**
- Compiler
- Assembler
  - Machine Code
  - Object File Format
- Linker
  - Resolve References
- Loader
- Program Translation vs. Interpretation

Steps in Compiling a Program



- "**Compiling C code to binary**" generally/**colloquially** refers to translating a C program to an executable.
- So far: `gcc foo.c`
  …produces `a.out`.

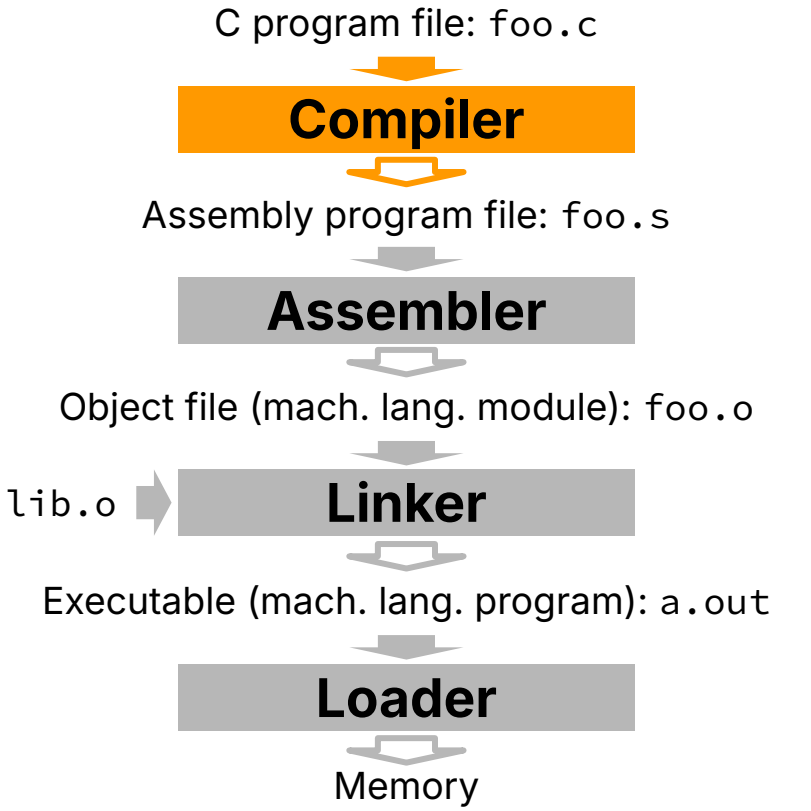
Steps in Compiling a Program



- "**Compiling C code to binary**" generally/**colloquially** refers to translating a C program to an executable.
- So far: `gcc foo.c`
  …produces `a.out`.

- This translation process is actually **3 steps**:
  - Compile, Assemble, Link
  - …followed by Load to execute

Steps in Compiling a Program

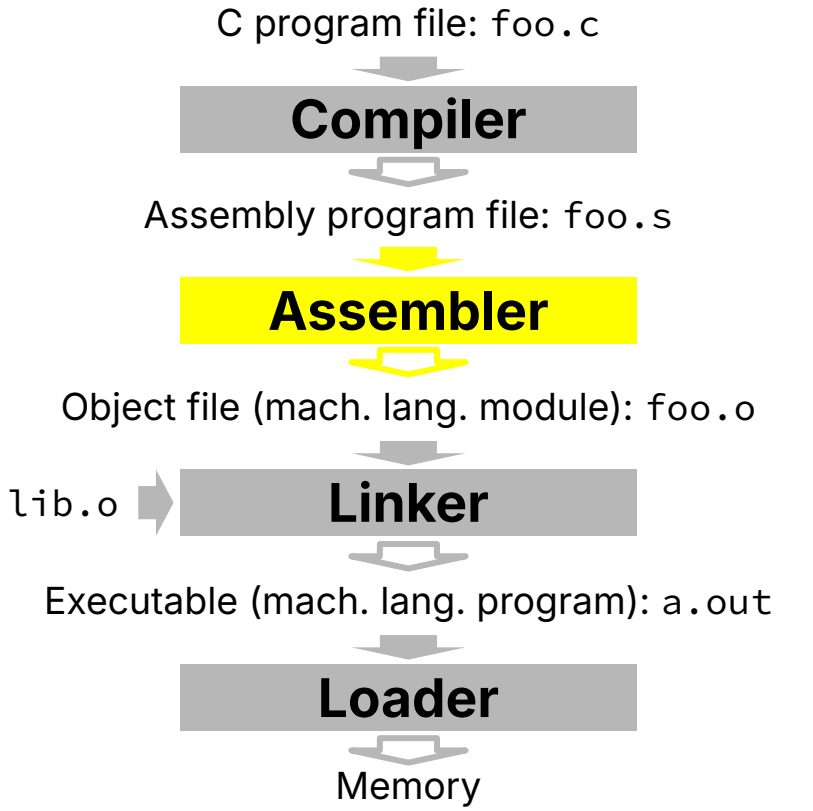


- Translates code to assembly language
- Input: High-level Language Code

  (e.g., **`foo.c`**)
- Output: Assembly Language Code

  (e.g., **`foo.s`** for RISC-V)
- Note: Output may contain pseudoinstructions!
  - e.g., `mv`, `li`, `call`, `j`, etc.

Steps in Compiling a Program



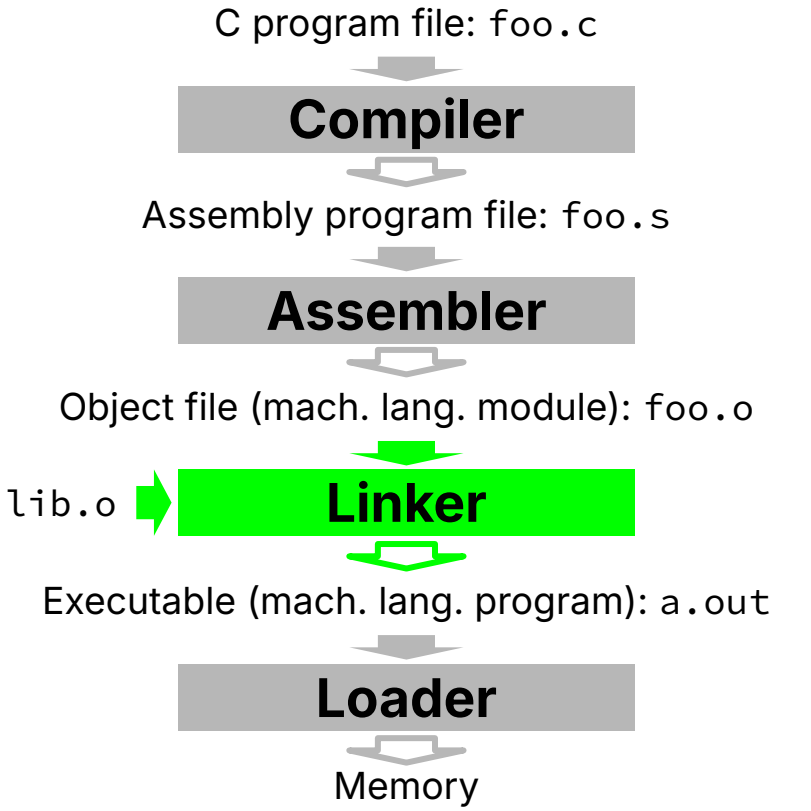
- Converts assembly code to binary
- Input: Assembly Language Code

  (e.g., **foo.s** for RISC-V)
- Output: Machine Language Module, object file

  (e.g., **foo.o** for RISC-V)
- Headers (libraries) are deferred to Linker

```
#include <stdio.h>
int main() {
    printf("Hello, %s\n",
           "world");
    return 0;
}
```

Steps in Compiling a Program



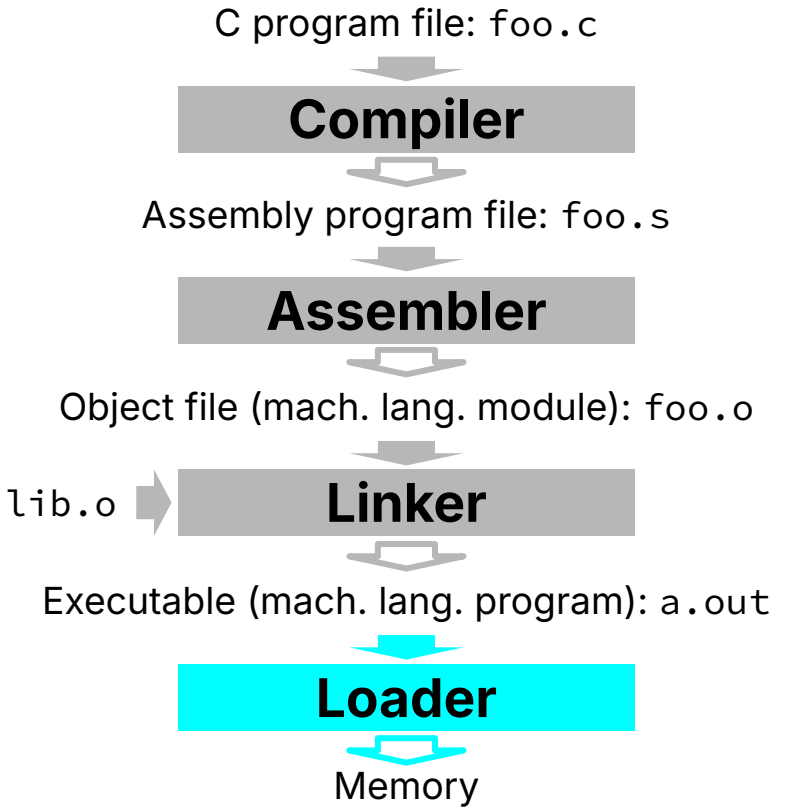
- Input: Object files

  (e.g., **`foo.o`**, **`lib.o`** for RISC-V)

- Output: Executable machine code
  (e.g., **`a.out`** for RISC-V)

- The Linker enables **separate** compilation of files.
  - Changes to one file **does not require recompilation** of the entire program.
    - C standard libraries, e.g., `stdio` (e.g., Linux source >20M lines of code)

Steps in Compiling a Program



- Input: Executable Code

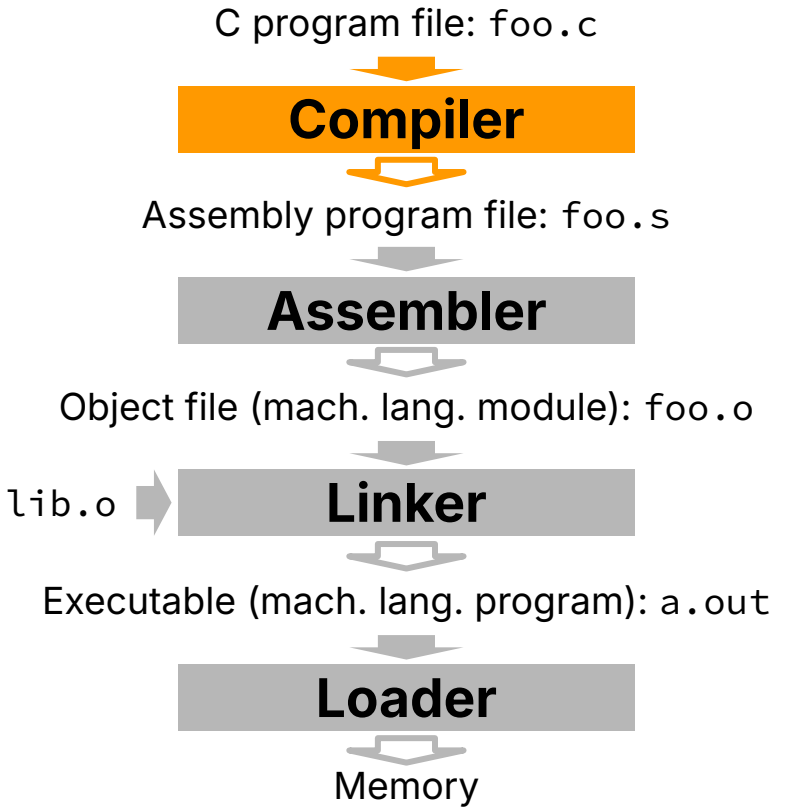
  (e.g., **`a.out`** for RISC-V)

  - Executable files are stored on disk.
- Output: (program is run)
  - When an executable is run, the loader first loads it into memory and then starts it running.

Agenda

- CALL: Compile, Assemble, Link, Load
- **Compiler**
- Assembler
  - Machine Code
  - Object File Format
- Linker
  - Resolve References
- Loader
- Example: Hello, World!
- Program Translation vs. Interpretation

Compiler, Detailed



- Translates code to assembly language
- Input: High-level Language Code

  (e.g., **`foo.c`**)
- Output: Assembly Language Code

  (e.g., **`foo.s`** for RISC-V)
- Note: Output may contain pseudoinstructions!
  - e.g., `mv`, `li`, `call`, `j`, etc.

Translating Code

- The most technically challenging step of CALL
  - Human language is very fluid, so many high-level languages also have built-in flexibility
  - Translating from a flexible high-level language requires a program that can "understand" flexible language–without benefiting from a Broca's region
- We won't go into too much detail about how compilation works; for more info, take CS 164 (and Linguistics classes)

Steps of Compilation

1. **Preprocessing**: Handle all preprocessor commands, macros
   - Technically happens before compiling starts, since preprocessing leaves you with valid C code
2. **Lexical Analysis**: Split code into "tokens" that can be given individual semantic meaning
   - E.g. split `int main()` into `["int", "main", "(", ")"]` instead of `["in", "t ma", "in(", ")"]`
3. **Semantic Analysis**: Parse the sentence structure so that the meaning of the sentence can be determined
   - E.g. Does a "*" token mean multiply, or dereference? Often uses a syntax tree.
4. **Optimization**: Optional, attempt to rewrite input code to be more efficient
   - Optimization level can be adjusted with compiler flags
5. **Encoding**: Generate code in a new language

Example: Hello World

Let's try to compile the following C program to RISC-V. We'll come back to this as we go through each step of CALL:

```
#include <stdio.h>
#define ITERS 10
int main() {
    for(int i = 0;i < ITERS;i++)
        printf("Hello %s\n","world");
    return 0;
}
```

Preprocessor


```
#include <stdio.h>
#define ITERS 10
int main() {
    for(int i = 0;i < ITERS;i++)
        printf("Hello %s\n","world");
    return 0;
}
```

Preprocessor


```
int main() {
    for(int i = 0;i < 10;i++)
        printf("Hello %s\n","world");
    return 0;
}
```

Lexer


```
int main ( ) { for ( int i = 0 ; i < 10 ; i ++ ) printf (
"Hello %s\n" , "world" ) ; return 0 ; }
```

Parser


```
Function: main
        type: int
        params: []
        variables: [
                variable:
                        type: int
                        name: i
        ]
        body: [
                for:
                        init: assign:
                                variable: i
                                value: 0
                        cond: <:
                                left: variable: i
                                right: constant: 10
                        after: postincrement:
                                operand: variable: i
                        body: [
                                Function call: printf
                                type: int
                                params: [
                                        Static String: "Hello %s\n",
                                        Static String: "world"
                                ]
                        ],
                return:
                        value: constant: 0
        ]
```

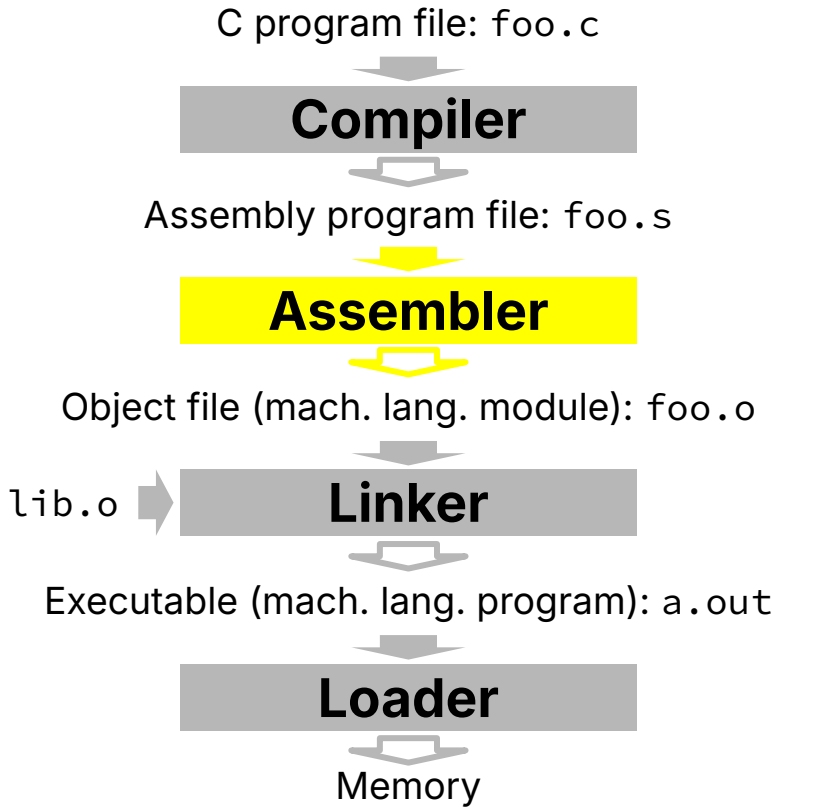
Encoder


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

Agenda

- CALL: Compile, Assemble, Link, Load
- Compiler
- **Assembler**
  - **Machine Code**
  - Object File Format
- Linker
  - Resolve References
- Loader
- Program Translation vs. Interpretation

Assembler, Detailed



- Converts assembly code to binary
- Input: Assembly Language Code

  (e.g., **`foo.s`** for RISC-V)
- Output: Machine Language Module, object file
  (e.g., **`foo.o`** for RISC-V)
- **Replace pseudoinstructions** with true assembly instructions.
- **Produce an object file** (a machine language **module**) that contains both this module's instructions and information for downstream steps (e.g., linking) and debugging.

Converting Instructions into Binary

- Most instructions have all necessary information within the instruction
- Pseudoinstructions have well-defined conversions to regular instructions.
- **Position-independent code** can also be computed
  - PC-Relative Branches and Jumps, e.g., beq/bne/etc. and jal
  - Once pseudoinstructions are replaced with real ones upstream (by Compiler), all **known** PC-relative addressing can be computed
  - Determine the offset to encode by counting the number of half-word instructions between current instruction and target instruction

```
add  x18 x18 x10
addi x19 x19  -1
```

```
j    Label
```

```
jal  x0 Label
```

bne	rs1 rs2 label	if (rs1 != rs2) PC = PC + offset
jal	rd label	rd = PC + 4 PC = PC + offset
jalr	rd rs1 imm	rd = PC + 4 PC = rs1 + imm

PC-Relative Addresses: 2-Pass

- The assembler performs **two passes** over the program to compute all offsets.
- "Forward Reference" problem:
  - Branches, PC-relative jumps can refer to labels that are "**forward**" in the program.
  - Precise **positive offsets** are unknown in the first pass!
- Instead, **take two passes** over the program.
  - Pass 1: Remember positions of labels (store in **symbol table**).
  - Pass 2: Use label positions to generate machine code.

```
        addi t2 zero 9
Loop:   slt  t1 zero t2
        beq  t1 zero Exit
        addi t2 t2 -1
        j Loop
Exit:   …
```

forward jump

Absolute Addresses? Not Yet

- References to **other files**?
  - e.g., call(`jalr`)-ing `strlen` from the C string library
- References to **static data**?
  - e.g., `la` gets broken up into `lui` and `addi`
  - These require knowing the full 32-bit address of the data
- **Absolute addresses** cannot be determined yet, so the Assembler records unresolved references in two tables:
  - Relocation Table and Symbol Table.

Assembler defers to downstream (Linker) any references that depend on knowing **other** object modules.

Agenda

- CALL: Compile, Assemble, Link, Load
- Compiler
- **Assembler**
  - Machine Code
  - **Object File Format**
- Linker
  - Resolve References
- Loader
- Program Translation vs. Interpretation

Directives: Information for Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

Directives give directions to the assembler:

- Often generated by the compiler (previous stage)
- **Directives do not produce machine instructions**! Rather, they inform for downstream how to build parts of the **output object file**.

`.text` …	Put subsequent items in the user Text segment (machine code)
`.data` …	Put subsequent items in user Data segment (source file data in binary)
`.globl sym`	Declares sym global and can be referenced from other files
`.string str`	Store the string str in memory and null-terminate it
`.word`	Store the n 32-bit quantities in successive memory words

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

Text:

Symbol Table		Relocation Table	

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

Text:
(aligned to multiple of 2**2 = 4)

Symbol Table		Relocation Table	

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

Text:
(aligned to multiple of 2**2 = 4)

Symbol Table		Relocation Table	
main (Global)			

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

Text:
0: addi sp sp -4
(Actually, the binary of this instruction)

Symbol Table		Relocation Table	
main (Global)	0(Text)		

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
0: addi sp sp -4
4: sw  ra 0 (sp)
```

Symbol Table		Relocation Table	
main (Global)	0(Text)		

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
0: addi sp sp -4
4: sw  ra 0 (sp)
8: addi t0 x0 10
```

Symbol Table		Relocation Table	
main (Global)	0(Text)		

```
Data:
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)		

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????
		18+1C: str2	????????

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????
		18+1C: str2	????????
		20+24: printf	????????

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????
		18+1C: str2	????????
		20+24: printf	????????

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????
		18+1C: str2	????????
		20+24: printf	????????
		2C: Loop	?????

```
Data:
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
		20+24: printf	????????
		2C: Loop	?????

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
34: addi sp sp 4
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
		20+24: printf	????????
		2C: Loop	?????

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
		20+24: printf	????????
		2C: Loop	?????

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
		20+24: printf	????????
		2C: Loop	?????

Data:

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
		20+24: printf	????????
		2C: Loop	?????

```
Data:
(Read-Only data aligned
to 4 bytes)
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
str1 (Static)	0(Data)	20+24: printf	????????
		2C: Loop	?????

```
Data:
0: "Hello, %s!\n"
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
str1 (Static)	0(Data)	20+24: printf	????????
str2 (Static)	D(Data)	2C: Loop	?????

```
Data:
0: "Hello, %s!\n"
D: "world"
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 ???
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	???
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
str1 (Static)	0(Data)	20+24: printf	????????
str2 (Static)	D(Data)	2C: Loop	?????

```
Data:
0: "Hello, %s!\n"
D: "world"
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	0x024
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
str1 (Static)	0(Data)	20+24: printf	????????
str2 (Static)	D(Data)	2C: Loop	?????

```
Data:
0: "Hello, %s!\n"
D: "world"
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	0x024
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
str1 (Static)	0(Data)	20+24: printf	????????
str2 (Static)	D(Data)	2C: Loop	?????

```
Data:
0: "Hello, %s!\n"
D: "world"
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	0x024
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
str1 (Static)	0(Data)	20+24: printf	????????
str2 (Static)	D(Data)	2C: Loop	?????

```
Data:
0: "Hello, %s!\n"
D: "world"
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	0x024
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
str1 (Static)	0(Data)	20+24: printf	????????
str2 (Static)	D(Data)	2C: Loop	?????

```
Data:
0: "Hello, %s!\n"
D: "world"
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 ?????
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	0x024
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
str1 (Static)	0(Data)	20+24: printf	????????
str2 (Static)	D(Data)	2C: Loop	?????

```
Data:
0: "Hello, %s!\n"
D: "world"
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	0x024
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
str1 (Static)	0(Data)	20+24: printf	????????
str2 (Static)	D(Data)	2C: Loop	0xFFFE0

```
Data:
0: "Hello, %s!\n"
D: "world"
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	0C: End	0x024
Loop (Local)	0C(Text)	10+14: str1	????????
End (Local)	30(Text)	18+1C: str2	????????
str1 (Static)	0(Data)	20+24: printf	????????
str2 (Static)	D(Data)	2C: Loop	0xFFFE0

```
Data:
0: "Hello, %s!\n"
D: "world"
```

Running the Assembler


```
.text
  .align 2
  .globl main
main:
  addi sp,sp,-4
  sw   ra,0(sp)
  li t0 10
Loop:
  beq t0 x0 End
  la   a0, str1
  la   a1, str2
  call printf
  addi t0 t0 -1
  j Loop
End:
  lw   ra,0(sp)
  addi sp,sp,4
  li   a0,0
  ret
.section .rodata
  .balign 4
str1:
  .string "Hello, %s!\n"
str2:
  .string "world"
```

```
Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
```

Symbol Table		Relocation Table	
main (Global)	00(Text)	10+14: str1	????????
str1 (Static)	0(Data)	18+1C: str2	????????
str2 (Static)	D(Data)	20+24: printf	????????

```
Data:
0: "Hello, %s!\n"
D: "world"
```

Object File Format

1. **Object File Header**: size and position of other pieces of the object file
2. **Text Segment**: machine code
3. **Data Segment**: binary representation of static data in the source file
4. **Symbol Table**: List of file's labels, static data that can be referenced by other programs
5. **Relocation Table**: Lines of code to fix later (by Linker)
6. **Debugging Information**

"table of contents"

Resolves PC-relative addresses

For downstream use

Common Standard Format: ELF (UNIX)
http://www.skyfree.org/linux/references/ELF_Format.pdf

Symbol and Relocation Table

Symbol Table:
List of “items” in this object file

- Also used by debugger (gdb)!
- Instruction labels
  - Used to compute machine code for PC-relative addressing in branches, function calling, etc.
  - Can use .globl **directive**: labels can be referenced by other files.
- **Data segment**: anything in section marked by the **.data** directive
  - Global variables may be accessed/used by other files.

Relocation Table:
List of “TODO items” whose address this file still needs

Any **external label** jumped to:

- e.g., in lib files
- `jal ext_label`

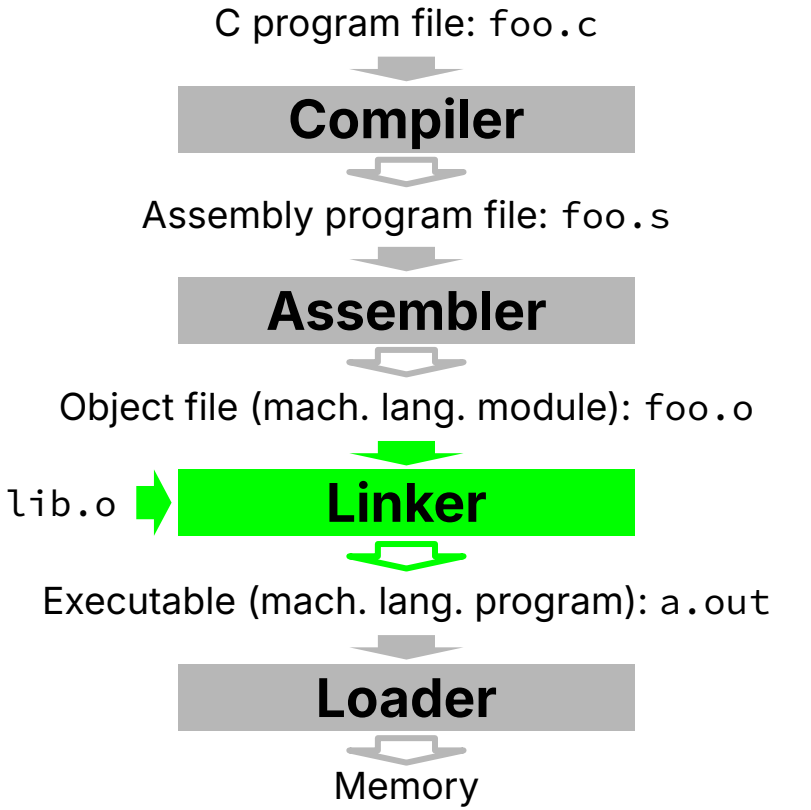
Any data located in **data segment**

- e.g., static variables
- `la` load address instruction

Agenda

- CALL: Compile, Assemble, Link, Load
- Compiler
- Assembler
  - Machine Code
  - Object File Format
- **Linker**
  - **Resolve References**
- Loader
- Program Translation vs. Interpretation

Linker, Detailed



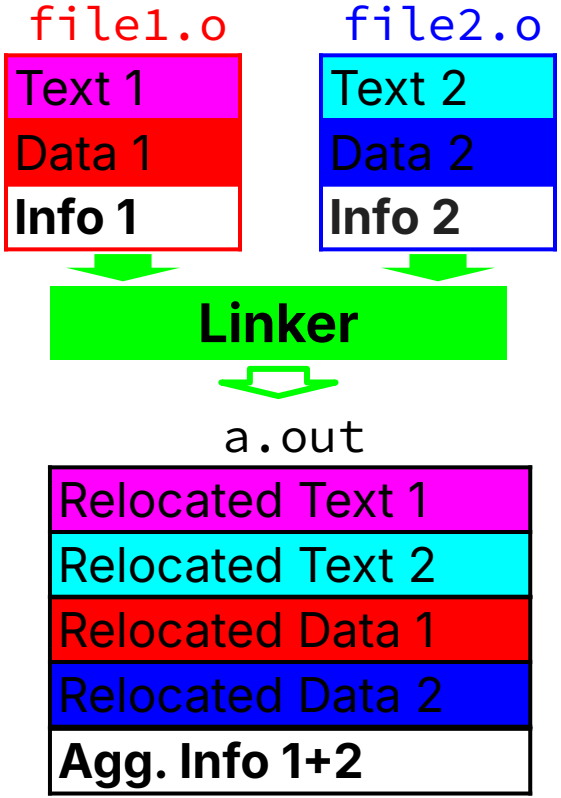
- Input: Object files

  (e.g., **`foo.o`**, **`lib.o`** for RISC-V)

- Output: Executable machine code
  (e.g., **`a.out`** for RISC-V)

- The Linker enables **separate** compilation of files.
  - Changes to one file **does not require recompilation** of the entire program.
    - C standard libraries, e.g., `stdio` (e.g., Linux source >20M lines of code)

Linking Multiple Assembly Files



1. Put together **text segments** from each .o file.
2. Put together **data segments** from each .o file, then concatenate this onto the end of Step 1's segment.
3. Resolve references.
   - Go through **Relocation Table** and handle each entry, i.e., fill in/update all **absolute addresses** to external labels and static data.

The linker knows all input object file modules, so it can resolve all the assembler's "TODO items" that require relocating for the final executable.

Resolving References

- For RV32, linker assumes first text segment starts at address `0x10000`.
  - (More later: virtual memory)
- Linker knows:
  - Length of each text and data segment
  - Ordering of text and data segments
- Linker calculates:
  - Absolute address of each label to be jumped to and of each piece of data referenced.

- Linker resolves references:
  - Search for reference (data or label) in all “user” symbol tables.
  - If not found, search library files (e.g., for `printf`).
  - Once absolute address determined, fill in machine code appropriately.

- Linker Output:
  - Executable containing text and data along with header/debugging info)

Running the Linker


```
HelloWorld Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
(0x40 bytes)

Data:
0: "Hello, %s!\n"
D: "world"

(0x20 bytes)
```

Symbol Table	
main (Global)	00(Text)
str1 (Static)	0(Data)
str2 (Static)	D(Data)

Relocation Table	
10+14: str1	????????
18+1C: str2	????????
20+24: printf	????????

<stdio>
(0x1000 bytes)

Symbol Table	
...	...
printf(Global)	800(Data)
...	...

Running the Linker


```
HelloWorld Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
(0x40 bytes)

Data:
0: "Hello, %s!\n"
D: "world"

(0x20 bytes)
```

Symbol Table	
main (Global)	00(Text)
str1 (Static)	0(Data)
str2 (Static)	D(Data)

Relocation Table	
10+14: str1	????????
18+1C: str2	????????
20+24: printf	????????

<stdio>
(0x1000 bytes)

Symbol Table	
...	...
printf(Global)	800(Data)
...	...

Linker Decides to Store Code At: (arbitrary)	
HelloWorld Text	0x00010000
stdio Text	0x00010100
HelloWorld Data	0x10008000
...	...

Running the Linker


```
HelloWorld Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 ?????
14: addi a0 a0 ???
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
(0x40 bytes)

Data:
0: "Hello, %s!\n"
D: "world"

(0x20 bytes)
```

Symbol Table	
main (Global)	00(Text)
str1 (Static)	0(Data)
str2 (Static)	D(Data)

Relocation Table	
10+14: str1	????????
18+1C: str2	????????
20+24: printf	????????

<stdio>
(0x1000 bytes)

Symbol Table	
...	...
printf(Global)	800(Data)
...	...

Linker Decides to Store Code At: (arbitrary)	
HelloWorld Text	0x00010000
stdio Text	0x00010100
HelloWorld Data	0x10008000
...	...

Running the Linker


```
HelloWorld Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 0x10008
14: addi a0 a0 0
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
(0x40 bytes)

Data:
0: "Hello, %s!\n"
D: "world"

(0x20 bytes)
```

Symbol Table	
main (Global)	00(Text)
str1 (Static)	0(Data)
str2 (Static)	D(Data)

Relocation Table	
10+14: str1	0x10008000
18+1C: str2	????????
20+24: printf	????????

<stdio>
(0x1000 bytes)

Symbol Table	
...	...
printf(Global)	800(Data)
...	...

Linker Decides to Store Code At: (arbitrary)	
HelloWorld Text	0x00010000
stdio Text	0x00010100
HelloWorld Data	0x10008000
...	...

Running the Linker


```
HelloWorld Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 0x10008
14: addi a0 a0 0
18: lui a1 ?????
1C: addi a1 a1 ???
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
(0x40 bytes)

Data:
0: "Hello, %s!\n"
D: "world"

(0x20 bytes)
```

Symbol Table	
main (Global)	00(Text)
str1 (Static)	0(Data)
str2 (Static)	D(Data)

Relocation Table	
10+14: str1	0x10008000
18+1C: str2	????????
20+24: printf	????????

<stdio>
(0x1000 bytes)

Symbol Table	
...	...
printf(Global)	800(Data)
...	...

Linker Decides to Store Code At: (arbitrary)	
HelloWorld Text	0x00010000
stdio Text	0x00010100
HelloWorld Data	0x10008000
...	...

Running the Linker


```
HelloWorld Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 0x10008
14: addi a0 a0 0
18: lui a1 0x10008
1C: addi a1 a1 0x00D
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
(0x40 bytes)

Data:
0: "Hello, %s!\n"
D: "world"

(0x20 bytes)
```

Symbol Table	
main (Global)	00(Text)
str1 (Static)	0(Data)
str2 (Static)	D(Data)

Relocation Table	
10+14: str1	0x10008000
18+1C: str2	0x1000800D
20+24: printf	????????

<stdio>
(0x1000 bytes)

Symbol Table	
...	...
printf(Global)	800(Data)
...	...

Linker Decides to Store Code At: (arbitrary)	
HelloWorld Text	0x00010000
stdio Text	0x00010100
HelloWorld Data	0x10008000
...	...

Running the Linker


```
HelloWorld Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 0x10008
14: addi a0 a0 0
18: lui a1 0x10008
1C: addi a1 a1 0x00D
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
(0x40 bytes)

Data:
0: "Hello, %s!\n"
D: "world"

(0x20 bytes)
```

Symbol Table	
main (Global)	00(Text)
str1 (Static)	0(Data)
str2 (Static)	D(Data)

Relocation Table	
10+14: str1	0x10008000
18+1C: str2	0x1000800D
20+24: printf	????????

<stdio>
(0x1000 bytes)

Symbol Table	
...	...
printf(Global)	800(Data)
...	...

Linker Decides to Store Code At: (arbitrary)	
HelloWorld Text	0x00010000
stdio Text	0x00010100
HelloWorld Data	0x10008000
...	...

Running the Linker


```
HelloWorld Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 0x10008
14: addi a0 a0 0
18: lui a1 0x10008
1C: addi a1 a1 0x00D
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
(0x40 bytes)

Data:
0: "Hello, %s!\n"
D: "world"

(0x20 bytes)
```

Symbol Table	
main (Global)	00(Text)
str1 (Static)	0(Data)
str2 (Static)	D(Data)

Relocation Table	
10+14: str1	0x10008000
18+1C: str2	0x1000800D
20+24: printf	0x10900-0x10020

<stdio>
(0x1000 bytes)

Symbol Table	
...	...
printf(Global)	800(Data)
...	...

Linker Decides to Store Code At: (arbitrary)	
HelloWorld Text	0x00010000
stdio Text	0x00010100
HelloWorld Data	0x10008000
...	...

Running the Linker


```
HelloWorld Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 0x10008
14: addi a0 a0 0
18: lui a1 0x10008
1C: addi a1 a1 0x00D
20: auipc ra ?????
24: jalr ra ra ???
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
(0x40 bytes)

Data:
0: "Hello, %s!\n"
D: "world"

(0x20 bytes)
```

Symbol Table	
main (Global)	00(Text)
str1 (Static)	0(Data)
str2 (Static)	D(Data)

Relocation Table	
10+14: str1	0x10008000
18+1C: str2	0x1000800D
20+24: printf	+0x0820

<stdio>
(0x1000 bytes)

Symbol Table	
...	...
printf(Global)	800(Data)
...	...

Linker Decides to Store Code At: (arbitrary)	
HelloWorld Text	0x00010000
stdio Text	0x00010100
HelloWorld Data	0x10008000
...	...

Running the Linker


```
HelloWorld Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 0x10008
14: addi a0 a0 0
18: lui a1 0x10008
1C: addi a1 a1 0x00D
20: auipc ra 1
24: jalr ra ra 0x820
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
(0x40 bytes)

Data:
0: "Hello, %s!\n"
D: "world"

(0x20 bytes)
```

Symbol Table	
main (Global)	00(Text)
str1 (Static)	0(Data)
str2 (Static)	D(Data)

Relocation Table	
10+14: str1	0x10008000
18+1C: str2	0x1000800D
20+24: printf	+0x0820

<stdio>
(0x1000 bytes)

Symbol Table	
...	...
printf(Global)	800(Data)
...	...

Linker Decides to Store Code At: (arbitrary)	
HelloWorld Text	0x00010000
stdio Text	0x00010100
HelloWorld Data	0x10008000
...	...

Running the Linker


```
HelloWorld Text:
00: addi sp sp -4
04: sw  ra 0 (sp)
08: addi t0 x0 10
0C: beq t0 x0 0x024
10: lui a0 0x10008
14: addi a0 a0 0
18: lui a1 0x10008
1C: addi a1 a1 0x00D
20: auipc ra 1
24: jalr ra ra 0x820
28: addi t0 t0 -1
2C: jal x0 0xFFFE0
30: lw ra 0(sp)
34: addi sp sp 4
38: addi x0 x0 0
3C: jalr x0 ra 0
(0x40 bytes)

Data:
0: "Hello, %s!\n"
D: "world"

(0x20 bytes)
```

Symbol Table	
main (Global)	00(Text)
str1 (Static)	0(Data)
str2 (Static)	D(Data)

Relocation Table	
10+14: str1	0x10008000
18+1C: str2	0x1000800D
20+24: printf	+0x0820

<stdio>
(0x1000 bytes)

Symbol Table	
...	...
printf(Global)	800(Data)
...	...

Linker Decides to Store Code At: (arbitrary)	
HelloWorld Text	0x00010000
stdio Text	0x00010100
HelloWorld Data	0x10008000
...	...

Aside: Static vs. Dynamic Linking

- So far, we've described the traditional way: **statically-linked libraries**.
  - The library is now **part of the executable** – if the library updates, we won't get the fix unless we recompile the user program.
  - The executable includes the entire library, even if not all of it is used by the user program.
  - Pro: The Executable is self-contained!
- The Alternative: **dynamically-linked libraries (DLL)**, common on Windows & UNIX Platforms.
  - The library is run as a separate "executable" when loading, and when a call is made to the library, code loaded from the library is run.

Aside: Dynamically-Linked Libraries (DLLs)

Space vs. Time

- Storing a program requires less disk space; less time to send a program.
- Executing two programs requires less memory (if they share a library)
- ⚠️ Runtime overhead: extra time to dynamically link!

Reasonable handling of upgrades

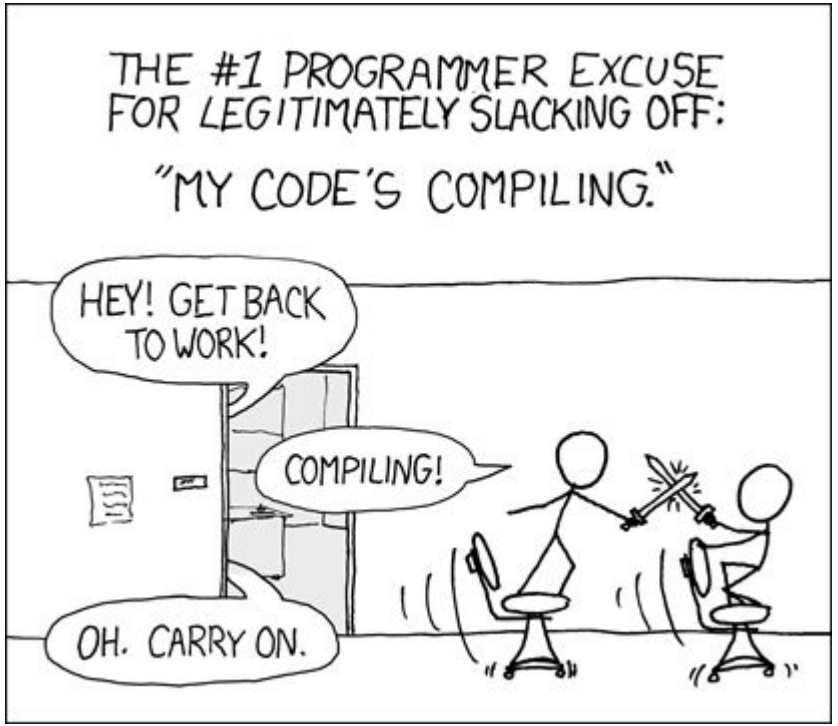
- Replacing `libXYZ.so` upgrades every program using library XYZ.
- ⚠️ Having the program executable isn't enough anymore!

Prevailing approach to DLL uses machine code as the "lowest common denominator."

- "Linking at the machine code level,"
  i.e., linker does not know what compiler/
  what language compiled from.
- Not the only the way to do it…

Overall dynamic linking **adds complexity** to compiler, linker and OS. However, its **benefits outweigh its complexities**.

Statically Compiled Code

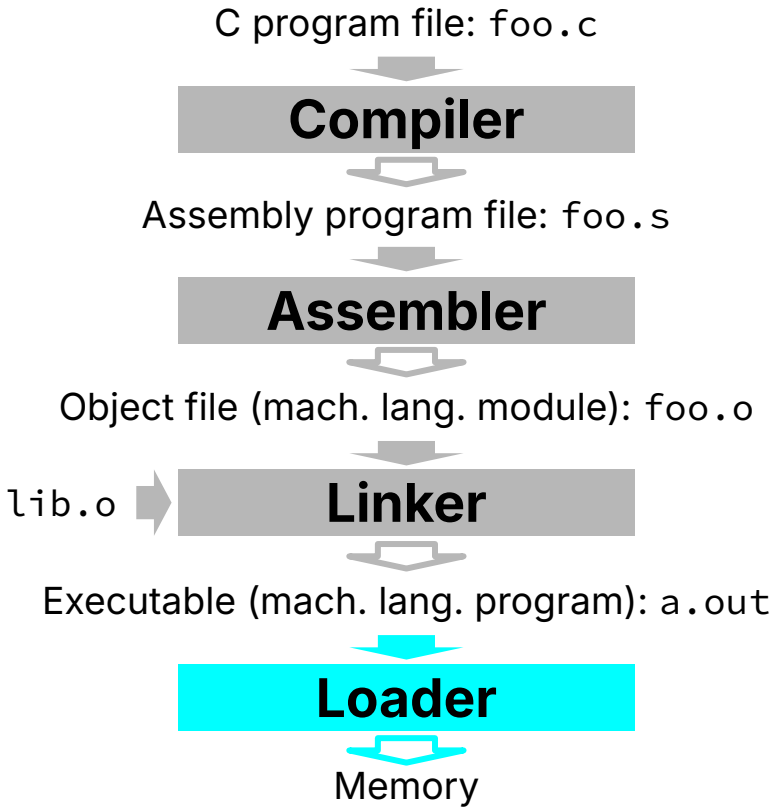


xkcd, explainxkcd

Agenda

- CALL: Compile, Assemble, Link, Load
- Compiler
- Assembler
  - Machine Code
  - Object File Format
- Linker
  - Resolve References
- **Loader**
- Program Translation vs. Interpretation

Loader, Detailed

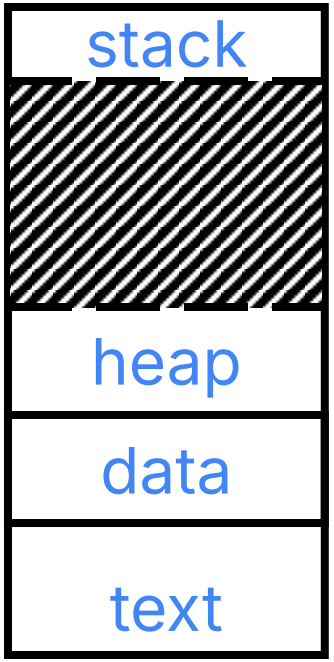


- Input: Executable Code

  (e.g., **`a.out`** for RISC-V)
  - Executable files are stored on disk.
- Output: (program is run)
  - When an executable is run, the loader first loads it into memory and then starts it running.
- Not really part of the compilation process
  - But part of the process of going from program to computer behavior
- Actually part of the *Operating System* (or OS)
  - More on this in Lecture 24, and CS 162

Loader

- Load program into a newly created address space:
  - Read `a.out` file header for sizes of text, data segments.
  - Create new address space for program large enough to hold **text and data segments**, along with a stack segment.
    - Copy instructions, data from executable file into new address space.
    - Copy arguments passed to the program onto the stack.
- Initialize machine registers
  - Most registers cleared; stack pointer (sp) assigned address of first free stack location
- Jump to start-up routine (main method):
  - Copy program arguments from stack to registers, set PC
- If main routine returns, terminate program with exit system call.



The **executable `a.out`** contains both machine code **(text segment)** and static data **(data segment)**.

Agenda

- CALL: Compile, Assemble, Link, Load
- Compiler
- Assembler
  - Machine Code
  - Object File Format
- Linker
  - Resolve References
- Loader
- **Program Translation vs. Interpretation**

Great Idea #1: Abstraction (Levels of Representation/Interpretation)

High(er) level language: C, C++, Rust

Assembly Language Programs: RISC-V, x86

Machine Language Program: RISC-V

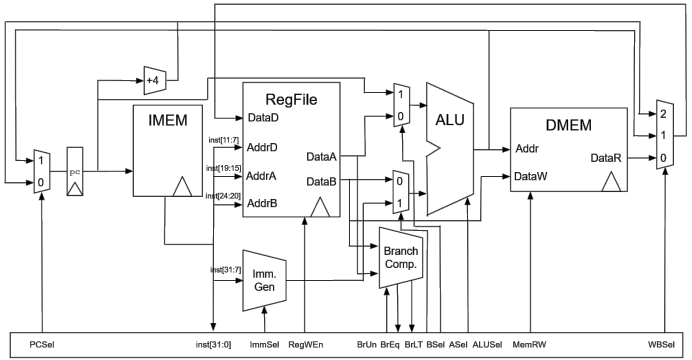
Hardware Architecture Description: block diagrams, hardware description language (HDL)

Logic Circuit Description: circuit schematic diagram, HDL

```
temp = V[k];
V[k] = V[k + 1];
V[k + 1] = temp;
```

```
la t0, V
lw t1, 0(t0)
amoswap.w.aq t1, t1, 4(t0)
sw t1, 0(t0)
```

Machine Language Program: RISC-V



Abstraction (Levels of Representation/Interpretation)

High(er) level language: C, C++, Rust

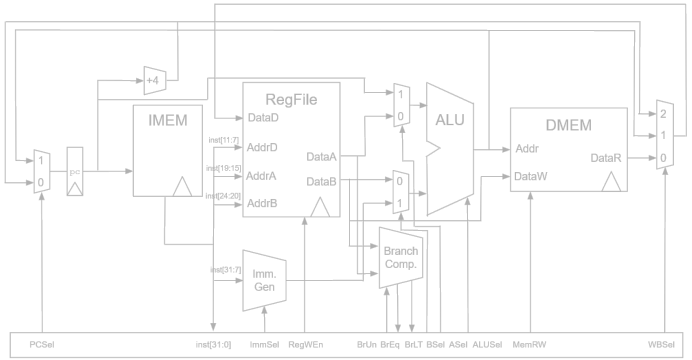
```
temp = V[k];
V[k] = V[k + 1];
V[k + 1] = temp;
```

Assembly Language Programs: RISC-V, x86

```
la t0, V
lw t1, 0(t0)
amoswap.w.aq t1, t1, 4(t0)
sw t1, 0(t0)
```

Machine Language Program: RISC-V

Machine Language Program: RISC-V



Hardware Architecture Description: block diagrams, hardware description language (HDL)

Logic Circuit Description: circuit schematic diagram, HDL

Abstraction (Levels of Representation/Interpretation)

High(er) level language: C, C++, Rust

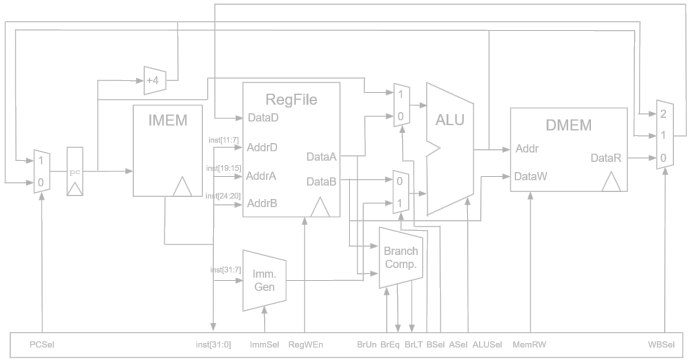
```
temp = V[k];
V[k] = V[k + 1];
V[k + 1] = temp;
```

Assembly Language Programs:
RISC-V, x86

```
la t0, V
lw t1, 0(t0)
amoswap.w.aq t1, t1, 4(t0)
sw t1, 0(t0)
```

Machine Language Program: RISC-V

Machine Language Program: RISC-V



Hardware Architecture Description:
block diagrams, hardware description language (HDL)

Logic Circuit Description: circuit schematic diagram, HDL

Abstraction (Levels of Representation/Interpretation)

High(er) level language: C, C++, Rust

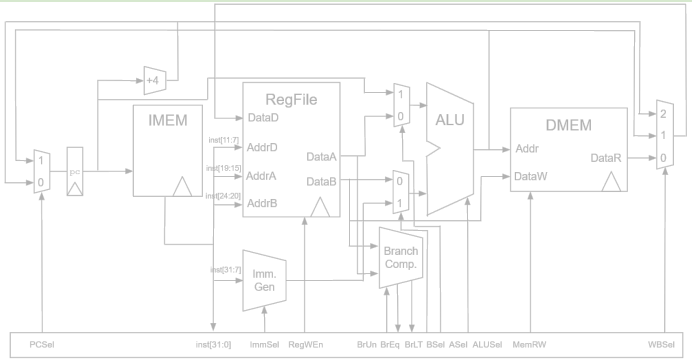
```
temp = V[k];
V[k] = V[k + 1];
V[k + 1] = temp;
```

Assembly Language Programs: RISC-V, x86

```
la t0, V
lw t1, 0(t0)
amoswap.w.aq t1, t1, 4(t0)
sw t1, 0(t0)
```

Machine Language Program: RISC-V

Machine Language Program: RISC-V

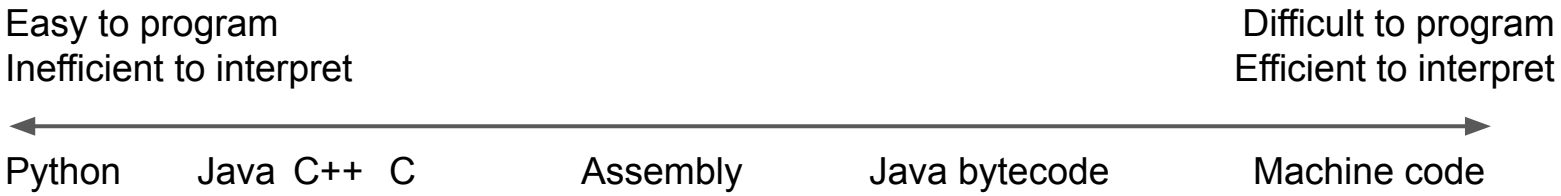


Hardware Architecture Description: block diagrams, hardware description language (HDL)

Logic Circuit Description: circuit schematic diagram, HDL

Language Execution Continuum

- **Interpreter** is a program that executes other programs
- Language **translation** gives us another option
- When to choose? In general, we
  - Interpret a high-level language when efficiency is not critical
  - Compile to a lower-level language to increase performance
  - Both are considered "translation"
  - Translate again to complete binary for system to run, in both

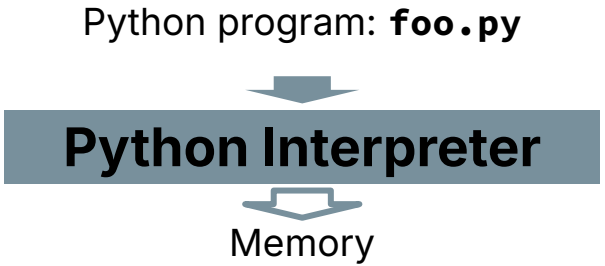


Interpretation vs. Translation

- How do we run a program written in a source language?
  - **Interpreter**: Directly executes a program in the source language
  - **Translator**: Converts a program from the source language to an equivalent program in another language

Interpretation

- Consider a Python program foo.py.
- Python interpreter is just a program that reads a python program and performs the functions of that python program



Interpretation

- WHY interpret machine language in software?
- E.g., VENUS RISC-V simulator useful for learning/debugging
- E.g., Apple Macintosh conversion
  - Switched from Motorola 680x0 ISA to PowerPC (before x86)
  - Could require all programs to be re-translated from high level language
  - Instead, let executables contain old and/or new machine code, interpret old code in software if necessary (emulation)

Interpretation vs. Translation (1/2)

- Generally easier to write interpreter
  - …you did it in CS61A!
- Interpreter closer to high-level, so can give better error messages (e.g., VENUS)
  - Translator reaction: add extra information to help debugging (line numbers, names)
- Interpreter slower (10x?), code smaller (2x?)
- Interpreter provides instruction set independence: run on any machine

Interpretation vs. Translation (2/2)

- Translated/compiled code almost always more efficient and therefore higher performance:
  - Important for many applications, particularly operating systems
- Translation/compilation helps “hide” the program “source” from the users:
  - One model for creating value in the marketplace (e.g., Microsoft keeps all their source code secret)
  - Alternative model, “open source”, creates value by publishing the source code and fostering a community of developers.